



Université Alioune Diop

Unité de Formation et de Recherche SATIC
Département TIC

PROJET DE FIN DE MODULE

CampusPulse

Système de gestion d'emploi du temps et d'alertes
universitaires en temps réel

Master : SI et SR
Module : Développement mobile
Groupe : 1

Présenté par :

Gana FAYE

Table des matières

1	Introduction générale	1
1.1	Contexte et problématique	1
1.2	Objectifs de CampusPulse	2
2	Analyse des besoins et cahier des charges	3
2.1	Analyse du cahier des charges initial	3
2.2	Besoins fonctionnels du MVP	4
2.3	Besoins non fonctionnels	4
3	Architecture technique et choix technologiques	6
3.1	Framework frontend : Flutter et langage Dart	6
3.2	Backend cloud : Firebase Realtime Database	6
3.3	Stockage local et cache : Hive	7
3.4	Architecture logicielle : Clean Architecture	7
4	Conception graphique et UI/UX (Figma)	9
4.1	Charte graphique et identité visuelle	9
4.1.1	Palette de couleurs	9
4.1.2	Typographie	9
4.2	Flux utilisateur et stratégie de navigation	10
4.3	Présentation des maquettes et captures d'écran	10
4.3.1	Écran d'accueil invité	10
4.3.2	Écran de connexion	11
4.3.3	Écran d'accueil après connexion	12
4.3.4	Écran d'emploi du temps	13
4.3.5	Écran des alertes	14
4.3.6	Vue globale des maquettes	15
4.4	Lien du prototype interactif Figma	16
5	Implémentation du code et logique métier	17
5.1	Structuration des modèles de données et tolérance aux valeurs nulles	17
5.1.1	Modèle utilisateur : <code>UserModel</code>	18
5.1.2	Modèle de cours : <code>CourseModel</code>	19
5.1.3	Modèle d'alerte : <code>AlertModel</code>	21
5.2	Gestion dynamique des sessions et auto-login avec Hive	23
5.3	Organisation de l'arborescence du projet (dossier <code>lib</code>)	24

5.3.1	Description des couches principales	24
5.3.2	Centralisation des services transverses	25
5.4	Algorithme de détection différentielle des changements de cours	25
5.5	Orchestration entre Firebase, Hive, Riverpod et l'interface	27
5.6	Synthèse de la logique métier implémentée	28
5.7	Lien du dépôt GitHub du projet	28
6	Réalisation de l'application et scénarios de test	29
6.1	Approche contextuelle de la réalisation	29
6.2	Déroulement chronologique du scénario utilisateur	30
6.2.1	Étape 1 : arrivée sur la page d'accueil invité	30
6.2.2	Étape 2 : transition vers l'écran de connexion	30
6.2.3	Étape 3 : validation de session et redirection vers l'accueil connecté .	31
6.2.4	Étape 4 : consultation de l'emploi du temps personnalisé	32
6.2.5	Étape 5 : interception d'un changement en temps réel et alerte visuelle	33
6.3	Bilan de la résilience en mode déconnecté	34
7	Mécanisme d'auto-login et persistance de session	35
7.1	Problématique de l'expérience utilisateur et objectif	35
7.2	Architecture technique et cycle de vie de la session	36
7.3	Implémentation logicielle et logique d'aiguillage	37
7.3.1	Persistance des jetons dans le service de session	37
7.3.2	Algorithme d'aiguillage dans le garde de GoRouter	37
7.4	Avantages métiers et résilience Offline-First	38
8	Conclusion et perspectives	40

Chapitre 1

Introduction générale

1.1 Contexte et problématique

Dans le cadre du projet de fin de module en développement d'application mobile, **CampusPulse** est proposée comme une application destinée aux étudiants de l'Université Alioune Diop de Bambey (UADB). Elle répond à un besoin essentiel : permettre l'accès rapide, fiable et centralisé à l'emploi du temps universitaire ainsi qu'aux alertes liées aux modifications de cours.

La gestion actuelle des emplois du temps reste confrontée à plusieurs difficultés. Les informations sont parfois dispersées entre les tableaux d'affichage, les groupes de discussion, les e-mails ou les annonces orales. De plus, les changements de salles, les reports ou les absences d'enseignants sont souvent communiqués tardivement. Cette situation peut provoquer des retards, des déplacements inutiles et une mauvaise organisation des étudiants.

Un autre problème majeur concerne la dépendance à la connexion internet. Dans un contexte où le réseau peut être instable sur le campus, une application qui exige une connexion permanente devient limitée. C'est pourquoi CampusPulse adopte une approche **Offline-First**, permettant à l'étudiant de consulter les dernières informations enregistrées même sans connexion active.

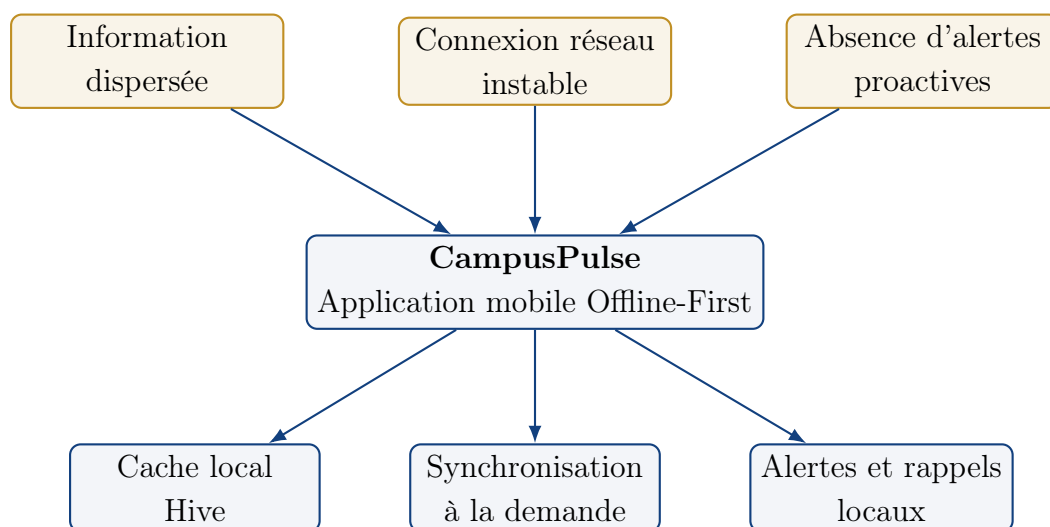


FIGURE 1.1 – Positionnement de CampusPulse face aux principales contraintes universitaires.

Ainsi, la problématique du projet peut être formulée comme suit : **comment concevoir une application mobile robuste, accessible hors ligne et capable d’informer rapidement les étudiants sur leur emploi du temps et ses éventuelles modifications ?**

1.2 Objectifs de CampusPulse

L’objectif général de CampusPulse est de fournir une solution mobile simple, fiable et évolutive pour améliorer l’accès aux informations pédagogiques. Même si l’application pourra intégrer plus tard d’autres services universitaires, cette première version se concentre principalement sur la **gestion de l’emploi du temps en temps réel**.

Les objectifs spécifiques sont résumés comme suit :

- proposer un agenda hebdomadaire clair, ergonomique et adapté à l’identité visuelle de l’UADB ;
- garantir la consultation de l’emploi du temps même sans connexion grâce au stockage local avec **Hive** ;
- permettre l’actualisation des données à la demande par un mécanisme de *Pull-to-refresh* ;
- notifier l’étudiant en cas de rappel de cours ou de modification importante ;
- organiser le code selon une architecture propre, séparant les couches *Data*, *Domain* et *Presentation*, avec une gestion d’état assurée par **Riverpod**.

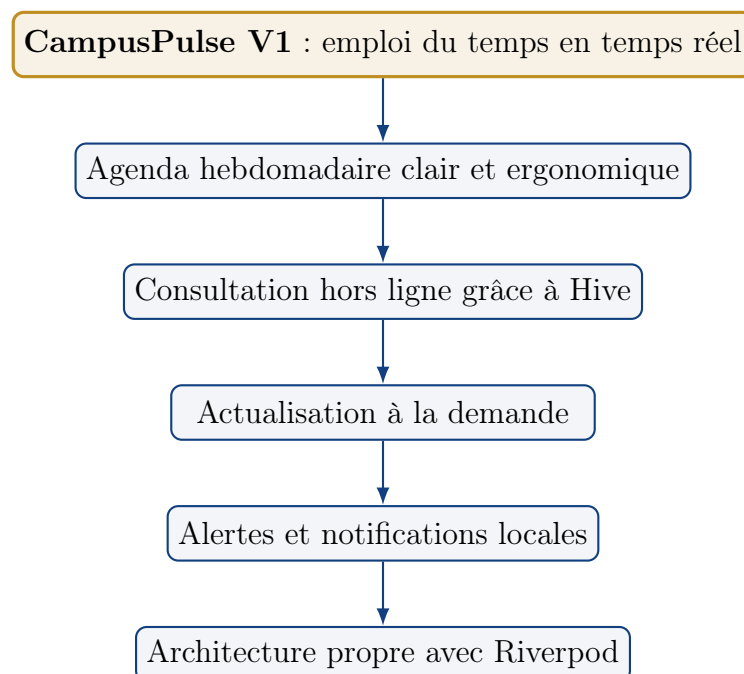


FIGURE 1.2 – Synthèse des objectifs de la première version de CampusPulse.

En résumé, CampusPulse vise à offrir aux étudiants une application pratique, réactive et adaptée aux réalités du campus, tout en constituant une base technique solide pour de futures évolutions.

Chapitre 2

Analyse des besoins et cahier des charges

2.1 Analyse du cahier des charges initial

Le cahier des charges initial du projet définit CampusPulse comme une solution mobile destinée à améliorer la communication académique autour des emplois du temps. L'analyse de ce document met en évidence une exigence principale : concevoir une application centrée sur l'étudiant, capable de fournir des informations pédagogiques fiables, actualisées et accessibles même lorsque la connexion internet est instable.

Le besoin identifié ne se limite pas à l'affichage d'un planning. Il s'agit de proposer un outil dynamique, capable de prendre en compte les changements fréquents du contexte universitaire : modification de salle, report de cours, absence d'enseignant ou annonce de dernière minute. L'application doit donc être pensée comme un produit mobile performant, et non comme une simple page web consultée depuis un téléphone.

À partir de cette analyse, le périmètre retenu pour la première version correspond à un **Produit Minimum Viable (MVP)**. Ce MVP regroupe les fonctionnalités essentielles permettant de tester la valeur du projet tout en respectant les contraintes de temps, de qualité et de maintenabilité.

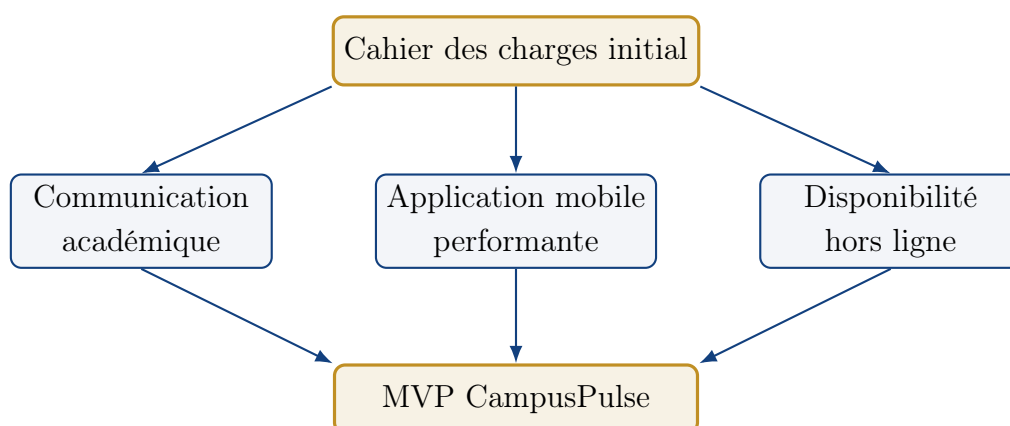


FIGURE 2.1 – Synthèse de l'analyse du cahier des charges initial.

2.2 Besoins fonctionnels du MVP

Les besoins fonctionnels décrivent les principales actions que l'étudiant doit pouvoir effectuer dans l'application. Pour cette première version, quatre fonctionnalités ont été retenues comme prioritaires.

- **Authentification unique et persistante** : l'étudiant se connecte avec son code étudiant. Après validation, sa session est conservée afin d'éviter une reconnexion à chaque ouverture de l'application.
- **Consultation de l'emploi du temps hebdomadaire** : l'utilisateur visualise les cours de la semaine sous forme de cartes contenant les informations essentielles : matière, enseignant, horaire et salle.
- **Actualisation à la demande** : un geste de type *Pull-to-refresh* permet de synchroniser les données locales avec la source distante lorsque la connexion est disponible.
- **Alertes et notifications locales** : l'application peut rappeler le début d'un cours ou signaler rapidement une modification importante, comme un changement de salle ou une annulation.

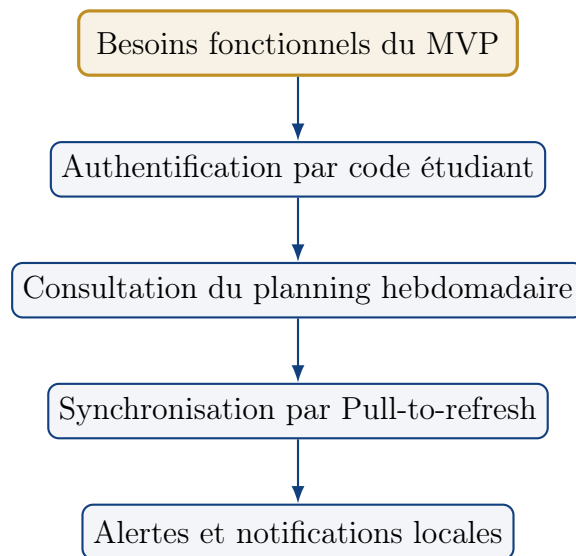


FIGURE 2.2 – Fonctionnalités essentielles retenues pour la première version.

2.3 Besoins non fonctionnels

Les besoins non fonctionnels précisent les exigences de qualité nécessaires pour garantir la fiabilité de CampusPulse. Ils concernent principalement la résilience réseau, la performance et la maintenabilité du code.

- **Résilience réseau et approche Offline-First** : l'application doit rester utilisable sans connexion. Les données récupérées lors d'une synchronisation réussie sont enregistrées localement avec Hive, puis réutilisées en cas d'indisponibilité du réseau.

- **Performance et fluidité de l'interface** : les opérations réseau, les lectures et écritures locales ainsi que les traitements de données doivent être exécutés de manière asynchrone afin de préserver la fluidité de l'interface.
- **Robustesse de l'architecture** : le code doit être structuré selon une organisation claire et évolutive. La séparation des couches *Data*, *Domain* et *Presentation* facilite la maintenance et l'ajout futur de nouveaux services.
- **Gestion rigoureuse de l'état** : l'utilisation de Riverpod permet de centraliser la logique d'état et d'éviter la présence de logique métier directement dans les écrans de l'application.

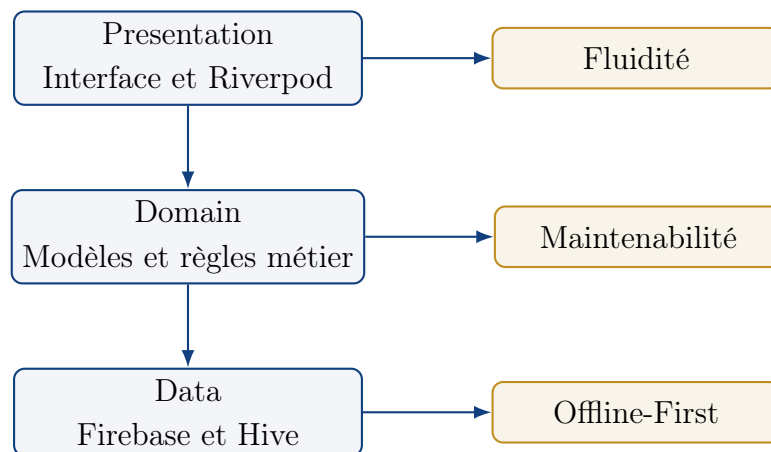


FIGURE 2.3 – Exigences non fonctionnelles et organisation technique attendue.

En résumé, l'analyse des besoins permet de fixer un cadre clair pour CampusPulse : une application mobile simple pour l'utilisateur, mais techniquement robuste, capable de fonctionner hors ligne, de synchroniser les données et de notifier l'étudiant au bon moment.

Chapitre 3

Architecture technique et choix technologiques

3.1 Framework frontend : Flutter et langage Dart

Le développement de **CampusPulse** repose sur **Flutter** et le langage **Dart**. Ce choix permet de créer une application multiplateforme à partir d'une seule base de code, utilisable aussi bien sur Android que sur iOS. Pour un projet universitaire, cette approche réduit le temps de développement tout en garantissant une interface cohérente sur différents smartphones.

Flutter offre une interface fluide grâce à son moteur de rendu graphique, tandis que Dart facilite l'écriture d'un code sûr et asynchrone. Les mécanismes **Future** et **Stream** sont particulièrement utiles pour gérer la synchronisation, les notifications et les données sans bloquer l'affichage.



FIGURE 3.1 – Flutter, framework retenu pour le développement mobile multiplateforme.

3.2 Backend cloud : Firebase Realtime Database

Le backend de CampusPulse s'appuie sur **Firebase Realtime Database**. Cette solution NoSQL permet de stocker les données sous forme d'arbre JSON et de diffuser rapidement les changements vers l'application mobile.

Ce choix est adapté au besoin de réactivité du projet. Lorsqu'une salle change ou qu'un cours est modifié, l'application peut recevoir la mise à jour sans attendre une vérification manuelle permanente. Firebase facilite donc la synchronisation en temps réel entre la base de données et l'interface de l'étudiant.



FIGURE 3.2 – Firebase, solution cloud utilisée pour la synchronisation temps réel.

3.3 Stockage local et cache : Hive

Pour assurer le fonctionnement **Offline-First**, l'application utilise **Hive**, une base locale NoSQL clé-valeur adaptée à Flutter. Hive permet de conserver les dernières données synchronisées afin que l'étudiant puisse consulter son emploi du temps même sans connexion internet.

Dans CampusPulse, Hive joue deux rôles essentiels : le stockage du planning en cache et la conservation de la session utilisateur. Ainsi, l'application peut afficher rapidement les dernières informations connues et éviter à l'étudiant de se reconnecter à chaque lancement.

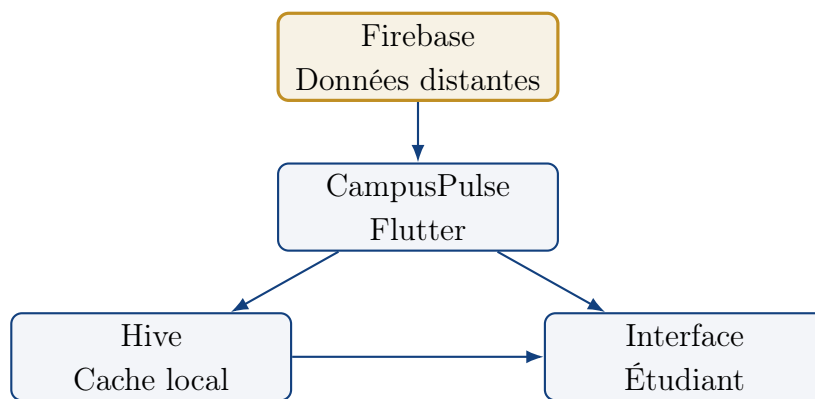


FIGURE 3.3 – Synchronisation entre Firebase, Flutter, Hive et l'interface utilisateur.

3.4 Architecture logicielle : Clean Architecture

Le projet adopte une organisation inspirée de la **Clean Architecture** afin de séparer les responsabilités. La couche **Domain** contient les modèles et règles métier, la couche **Data** gère Firebase et Hive, tandis que la couche **Presentation** regroupe les écrans Flutter et Riverpod. Cette organisation rend le code plus lisible et facilite les évolutions futures.

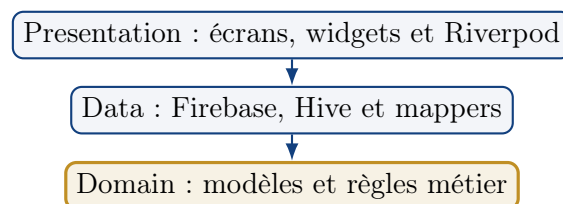


FIGURE 3.4 – Organisation simplifiée de la Clean Architecture adoptée.

En résumé, CampusPulse combine Flutter pour l'interface, Firebase pour la synchronisation, Hive pour le cache local et une Clean Architecture pour organiser le code.

Chapitre 4

Conception graphique et UI/UX (Figma)

4.1 Charte graphique et identité visuelle

La conception graphique de **CampusPulse** a été pensée pour proposer une interface à la fois institutionnelle, moderne et facile à utiliser. L’objectif est de créer un environnement visuel rassurant pour les étudiants de l’Université Alioune Diop de Bambey, tout en garantissant une consultation rapide des informations pédagogiques.

L’identité visuelle repose sur trois principes : la lisibilité, la cohérence et la hiérarchisation de l’information. Les écrans doivent permettre à l’étudiant d’identifier immédiatement les éléments importants : le jour sélectionné, le cours à venir, la salle, l’horaire et les éventuelles alertes.

4.1.1 Palette de couleurs

La palette retenue s’appuie sur des couleurs simples et fonctionnelles :

- **Bleu institutionnel** : utilisé pour les boutons principaux, les éléments actifs et certaines zones de navigation. Il rappelle l’identité universitaire et donne un aspect sérieux à l’application.
- **Rouge d’alerte** : réservé aux messages critiques comme les annulations, changements de salles ou notifications importantes.
- **Blanc et gris clair** : utilisés pour les arrière-plans et les cartes afin de préserver une lecture confortable sur smartphone.

4.1.2 Typographie

La typographie choisie est **Poppins**, une police sans-serif moderne et lisible. Les titres et intitulés de cours utilisent des graisses plus fortes, tandis que les informations secondaires, comme les horaires ou les noms d’enseignants, sont affichées avec une graisse plus légère. Cette hiérarchie typographique facilite la lecture rapide de l’emploi du temps.



FIGURE 4.1 – Palette graphique utilisée pour l’identité visuelle de CampusPulse.

4.2 Flux utilisateur et stratégie de navigation

L’expérience utilisateur de CampusPulse est organisée autour d’un parcours simple. L’application distingue d’abord l’utilisateur invité de l’étudiant connecté. Cette logique permet de présenter l’application au public tout en donnant à l’étudiant authentifié un accès direct à son emploi du temps personnalisé.

Le parcours principal est le suivant : l’utilisateur arrive sur l’écran d’accueil invité, accède à l’écran de connexion, saisit son code étudiant, puis rejoint son tableau de bord personnel. Lors des prochaines ouvertures, grâce à la session enregistrée localement, l’application peut afficher directement l’espace étudiant.

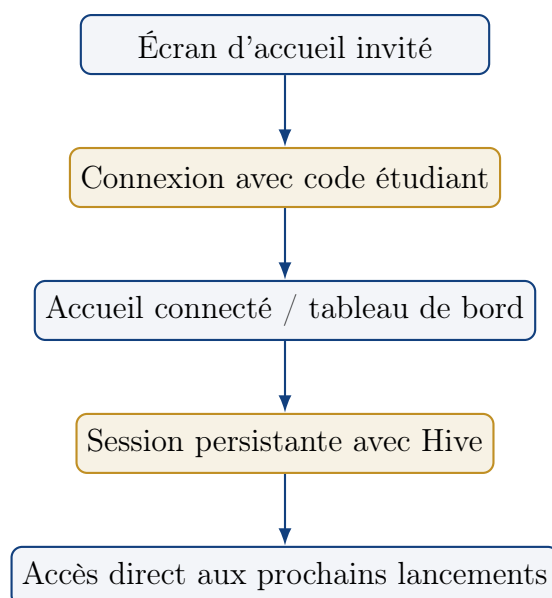


FIGURE 4.2 – Flux utilisateur principal de CampusPulse.

4.3 Présentation des maquettes et captures d’écran

La phase de maquettage réalisée sur Figma a permis de valider l’organisation visuelle des écrans avant le développement. Les maquettes ont servi à préciser les couleurs, les positions des composants, la navigation et la lisibilité des informations.

4.3.1 Écran d’accueil invité

L’écran d’accueil invité constitue la porte d’entrée de l’application. Il présente CampusPulse, affiche l’identité de l’université et invite l’utilisateur à se connecter pour accéder à un emploi du temps personnalisé.

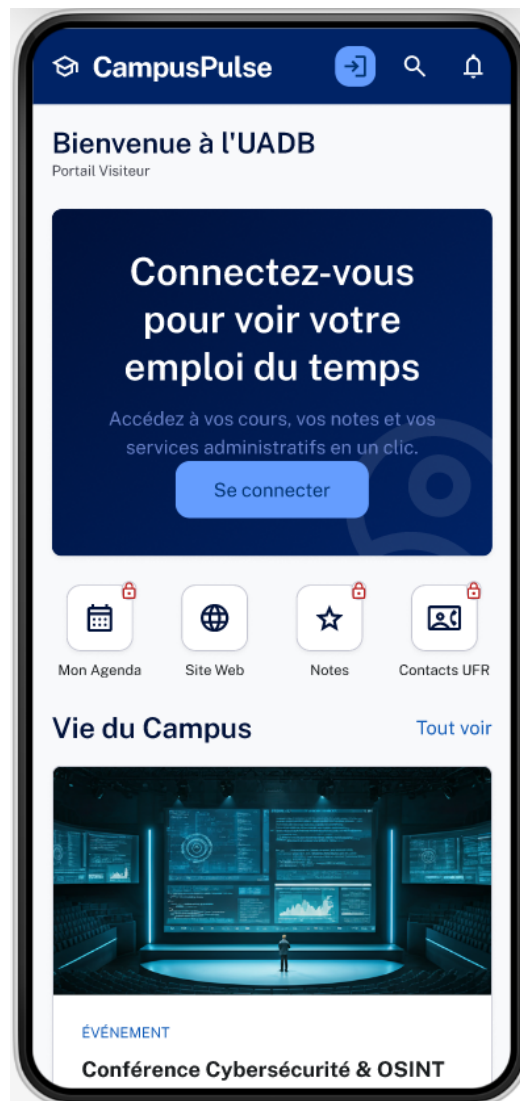


FIGURE 4.3 – Maquette haute-fidélité de l'écran d'accueil général en mode invité.

4.3.2 Écran de connexion

L'écran de connexion est volontairement minimaliste. Il se concentre sur une seule tâche : permettre à l'étudiant de saisir son code étudiant et d'accéder rapidement à son espace personnel.

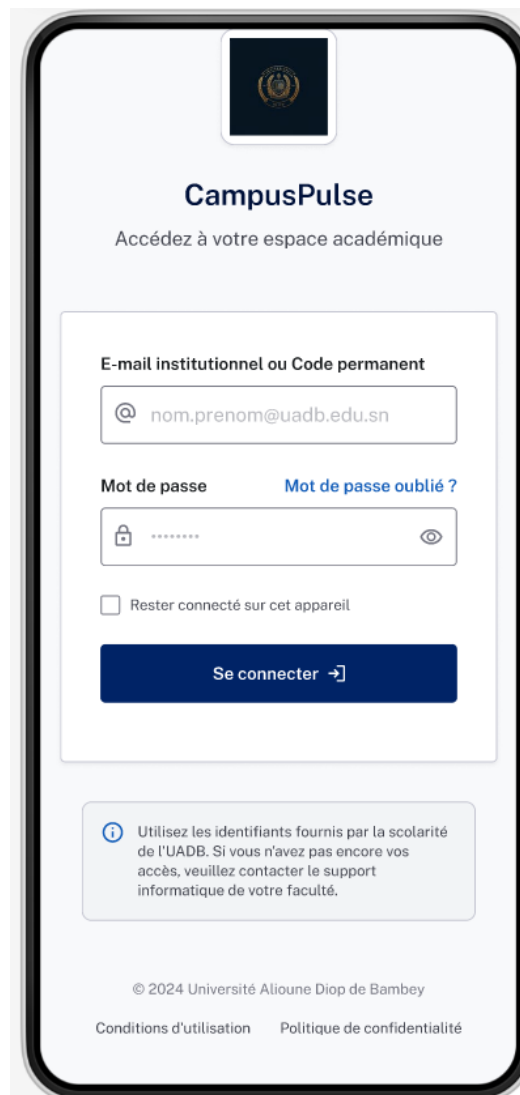


FIGURE 4.4 – Maquette haute-fidélité de l'écran d'authentification unique.

4.3.3 Écran d'accueil après connexion

Après authentification, l'étudiant accède à un écran personnalisé. Celui-ci présente les informations liées à son profil, son niveau et son espace de consultation. Cette interface constitue le point d'accès principal aux fonctionnalités de l'application.



FIGURE 4.5 – Maquette de l'écran d'accueil après connexion réussie.

4.3.4 Écran d'emploi du temps

L'écran d'emploi du temps représente le cœur fonctionnel de CampusPulse. Les cours sont organisés sous forme de cartes lisibles, avec les informations importantes : matière, horaire, enseignant et salle. L'étudiant peut consulter rapidement sa journée et suivre les changements éventuels.



FIGURE 4.6 – Vue de l’agenda hebdomadaire personnalisé de l’étudiant.

4.3.5 Écran des alertes

L’écran des alertes rassemble les notifications et changements importants liés au planning. Il permet à l’étudiant de repérer rapidement les modifications récentes, notamment les changements de salles, reports ou annulations.



FIGURE 4.7 – Interface de l'historique des alertes de planning en temps réel.

4.3.6 Vue globale des maquettes

Une vue globale du prototype permet de visualiser la cohérence générale entre les différents écrans conçus sur Figma.

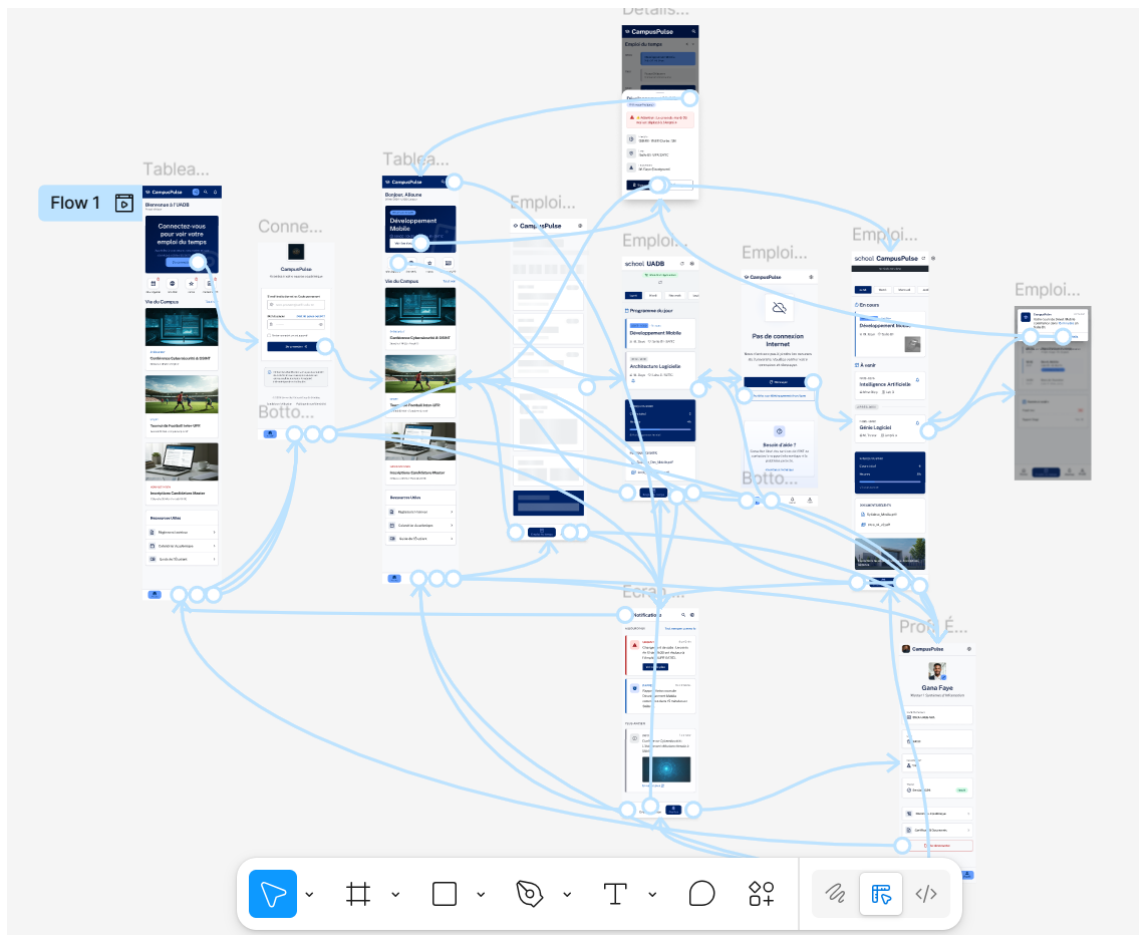


FIGURE 4.8 – Vue d’ensemble des maquettes Figma de CampusPulse.

4.4 Lien du prototype interactif Figma

Afin de simuler le comportement réel de l’application, un prototype interactif a été configuré sur Figma. Il permet de tester les transitions entre les écrans, notamment le passage de l’accueil invité vers l’écran de connexion, puis vers l’espace étudiant connecté.

Le prototype interactif est accessible à l’adresse suivante :

<https://www.figma.com/proto/g17XXcgrWTXVS3hdjSackr/CampusPuls?node-id=1-1155&starting-point-node-id=1%3A1155&t=2RxDZugszGe5cvtk-1>

En résumé, la conception UI/UX de CampusPulse vise à rendre l’application claire, cohérente et agréable à utiliser. Les maquettes Figma ont permis de fixer une base visuelle solide avant l’implémentation technique en Flutter.

Chapitre 5

Implémentation du code et logique métier

5.1 Structuration des modèles de données et tolérance aux valeurs nulles

Dans la couche **Domain** et la couche **Data** de l'application **CampusPulse**, la modélisation des données occupe une place centrale. Les informations manipulées par l'application proviennent principalement de **Firestore Realtime Database**. Elles transitent sous forme de structures JSON dynamiques, donc potentiellement incomplètes, mal typées ou temporairement incohérentes. Dans ce contexte, le premier objectif technique est d'empêcher qu'une valeur absente, une chaîne vide ou un type inattendu provoque un plantage de l'application sur le téléphone de l'étudiant.

La stratégie retenue consiste à créer des modèles robustes, capables de transformer les données brutes en objets Dart exploitables. Le modèle `CourseModel`, qui représente un cours dans l'emploi du temps, joue ce rôle de filtre. Il reçoit l'identifiant du cours ainsi qu'une structure `Map` issue de Firestore, puis applique des règles de sécurisation avant de construire l'objet final.

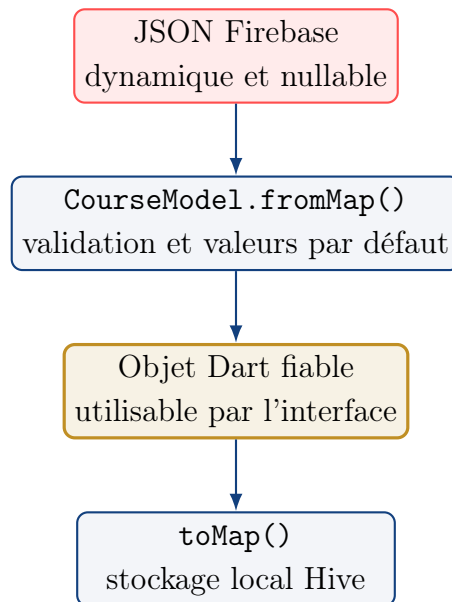


FIGURE 5.1 – Chaîne de transformation sécurisée entre Firebase, le modèle métier et Hive.

Le constructeur d'usine `fromMap` constitue ainsi le premier rempart de sécurité. Lorsqu'un champ comme le titre, l'enseignant ou la salle est absent de l'arbre JSON, le modèle le remplace par une valeur textuelle explicite. De même, les dates sont converties en objets `DateTime` afin que la logique métier puisse comparer les horaires sans manipuler directement des chaînes de caractères.

5.1.1 Modèle utilisateur : UserModel

Le modèle `UserModel` représente l'étudiant connecté. Il centralise les informations nécessaires à l'identification et au filtrage pédagogique : identifiant unique, nom, adresse e-mail, code étudiant, UFR et niveau. Son constructeur `fromMap` accepte plusieurs conventions de clés afin de rester compatible avec différentes structures Firebase, par exemple `name`, `fullName`, `nom`, `codeEtudiant`, `code_etudiant` ou encore `code`.

Listing 5.1 – Modèle réel de l'utilisateur étudiant.

```
class UserModel {  
  final String uid;  
  final String name;  
  final String email;  
  final String codeEtudiant;  
  final String ufr;  
  final String niveau;  
  
  UserModel({  
    required this.uid,  
    required this.name,  
    required this.email,  
    required this.codeEtudiant,  
    required this.ufr,  
    required this.niveau,  
  });  
}
```

```

factory UserModel.fromMap(Map<dynamic, dynamic> map, {String? id}) {
    final m = Map<String, dynamic>.from(map);
    return UserModel(
        uid: id ?? m['uid'] ?? m['id'] ?? '',
        name: m['name'] ?? m['fullName'] ?? m['nom'] ?? '',
        email: m['email'] ?? m['mail'] ?? '',
        codeEtudiant: m['codeEtudiant'] ?? m['code_etudiant'] ?? m['code'] ?? '',
        ufr: m['ufr'] ?? m['department'] ?? m['faculte'] ?? '',
        niveau: m['niveau'] ?? m['level'] ?? m['classe'] ?? '',
    );
}

Map<String, dynamic> toMap() {
    return {
        'uid': uid,
        'name': name,
        'email': email,
        'codeEtudiant': codeEtudiant,
        'ufr': ufr,
        'niveau': niveau,
    };
}
}

```

5.1.2 Modèle de cours : CourseModel

Le modèle `CourseModel` décrit chaque élément de l'emploi du temps. Il contient l'identifiant du cours, l'enseignant, la salle ou classe, les horaires de début et de fin, ainsi que les champs `ufr` et `niveau`. Ces deux derniers attributs sont essentiels pour afficher uniquement les cours correspondant au profil académique de l'étudiant connecté.

Listing 5.2 – Modèle réel d'un cours planifié.

```

class CourseModel {
    final String? studentUid;
    final String id;
    final String titre;
    final String professeur;
    final String classe;
    final DateTime debutTime;
    final DateTime finTime;
    final String ufr;
    final String niveau;

    CourseModel({
        this.studentUid,
        required this.id,
        required this.titre,
        required this.proesseur,
        required this.classe,
        required this.debutTime,
    }) {}
}

```

```

        required this.finTime,
        required this.ufr,
        required this.niveau,
    });

factory CourseModel.fromMap(Map<dynamic, dynamic> map, {String? id}) {
    final m = Map<String, dynamic>.from(map);

    DateTime parseTime(dynamic v) {
        if (v == null) return DateTime.fromMillisecondsSinceEpoch(0);
        if (v is int) return DateTime.fromMillisecondsSinceEpoch(v);
        if (v is String) {
            try {
                return DateTime.parse(v);
            } catch (_) {
                return DateTime.fromMillisecondsSinceEpoch(0);
            }
        }
        return DateTime.fromMillisecondsSinceEpoch(0);
    }

    return CourseModel(
        studentUid: m['studentUid'] ??
            m['student_uid'] ??
            m['student'] ??
            m['studentId'],
        id: id ?? m['id'] ?? m['courseId'] ?? '',
        titre: m['titre'] ?? m['title'] ?? m['name'] ?? '',
        professeur: m['professeur'] ?? m['teacher'] ?? m['enseignant'] ?? '',
        classe: m['classe'] ?? m['room'] ?? m['salle'] ?? '',
        debutTime: parseTime(
            m['debutTime'] ?? m['startTime'] ?? m['debut'] ?? m['start'],
        ),
        finTime: parseTime(
            m['finTime'] ?? m['endTime'] ?? m['fin'] ?? m['end'],
        ),
        ufr: m['ufr'] ?? m['department'] ?? '',
        niveau: m['niveau'] ?? m['level'] ?? '',
    );
}

Map<String, dynamic> toMap() {
    return {
        'studentUid': studentUid,
        'id': id,
        'titre': titre,
        'professeur': professeur,
        'classe': classe,
        'debutTime': debutTime.toIso8601String(),
        'finTime': finTime.toIso8601String(),
        'ufr': ufr,
        'niveau': niveau,
    };
}

```

```
}  
}
```

La fonction interne `parseTime` rend le modèle plus robuste : elle accepte aussi bien un entier représentant un timestamp qu'une chaîne au format ISO-8601. Si la valeur reçue est absente ou invalide, elle retourne une date neutre afin d'éviter une exception pendant la construction de l'objet.

5.1.3 Modèle d'alerte : `AlertModel`

Le modèle `AlertModel` structure les notifications affichées à l'étudiant. Il distingue trois catégories : `urgent`, `rappel` et `info`. Cette catégorisation permet ensuite à l'interface d'adapter la couleur, l'icône et la priorité d'affichage selon la nature du message.

Listing 5.3 – Modèle réel d'une alerte ou notification.

```
// lib/data/models/alert_model.dart  
import 'package:flutter/material.dart';  
  
enum AlertType { urgent, rappel, info }  
  
class AlertModel {  
  final String id;  
  final AlertType type;  
  final String tag;  
  final String message;  
  final DateTime timestamp;  
  final String? actionLabel;  
  final VoidCallback? onAction;  
  
  AlertModel({  
    required this.id,  
    required this.type,  
    required this.tag,  
    required this.message,  
    required this.timestamp,  
    this.actionLabel,  
    this.onAction,  
  });  
  
  factory AlertModel.fromMap(Map<dynamic, dynamic> map, {String? id}) {  
    final m = Map<String, dynamic>.from(map);  
  
    AlertType parseType(String? s) {  
      if (s == null) return AlertType.info;  
      final normalized = s.toLowerCase();  
      if (normalized.contains('urgent')) return AlertType.urgent;  
      if (normalized.contains('rappel') ||  
          normalized.contains('reminder')) {  
        return AlertType.rappel;  
      }  
      return AlertType.info;  
    }  
  }  
}
```

```

}

DateTime parseTimestamp(dynamic v) {
  if (v == null) return DateTime.now();
  if (v is int) return DateTime.fromMillisecondsSinceEpoch(v);
  if (v is String) {
    try {
      return DateTime.parse(v);
    } catch (_) {
      return DateTime.now();
    }
  }
  return DateTime.now();
}

return AlertModel(
  id: id ?? m['id'] ?? m['alertId'] ?? '',
  type: parseType(m['type'] ?? m['category'] ?? m['niveau']),
  tag: m['tag'] ?? m['title'] ?? m['titre'] ?? '',
  message: m['message'] ?? m['subtitle'] ?? m['body'] ?? '',
  timestamp: parseTimestamp(m['timestamp'] ?? m['time'] ?? m['date']),
  actionLabel: m['actionLabel'] ?? m['action_label'],
  onAction: null,
);
}

Map<String, dynamic> toMap() {
  return {
    'id': id,
    'type': type.toString().split('.').last,
    'tag': tag,
    'message': message,
    'timestamp': timestamp.toIso8601String(),
    'actionLabel': actionLabel,
  };
}
}

```

Grâce à cette séparation, chaque modèle possède une présentation claire : le modèle utilisateur pour l'identité, le modèle de cours pour le planning, et le modèle d'alerte pour les notifications.

Cette implémentation améliore la stabilité générale de l'application. Au lieu de faire confiance aux données distantes, CampusPulse applique une politique de défense systématique : toute donnée entrante est convertie, contrôlée puis normalisée avant d'être utilisée par l'interface ou stockée dans le cache local.

5.2 Gestion dynamique des sessions et auto-login avec Hive

La deuxième partie importante de l'implémentation concerne la gestion de session. Une application universitaire doit éviter de demander à l'étudiant de saisir son code à chaque ouverture. Pour cette raison, CampusPulse utilise **Hive** comme mécanisme de persistance locale. Hive permet d'enregistrer de petites informations clé-valeur de manière rapide, sans serveur et sans connexion internet.

Le service `SessionService` encapsule toute la logique de session. Il ouvre une boîte locale nommée `sessionBox`, y stocke le code étudiant après une authentification réussie, puis expose des méthodes simples pour vérifier si l'utilisateur est déjà connecté. Cette séparation évite de disperser les appels Hive dans les écrans Flutter et rend le code plus maintenable.

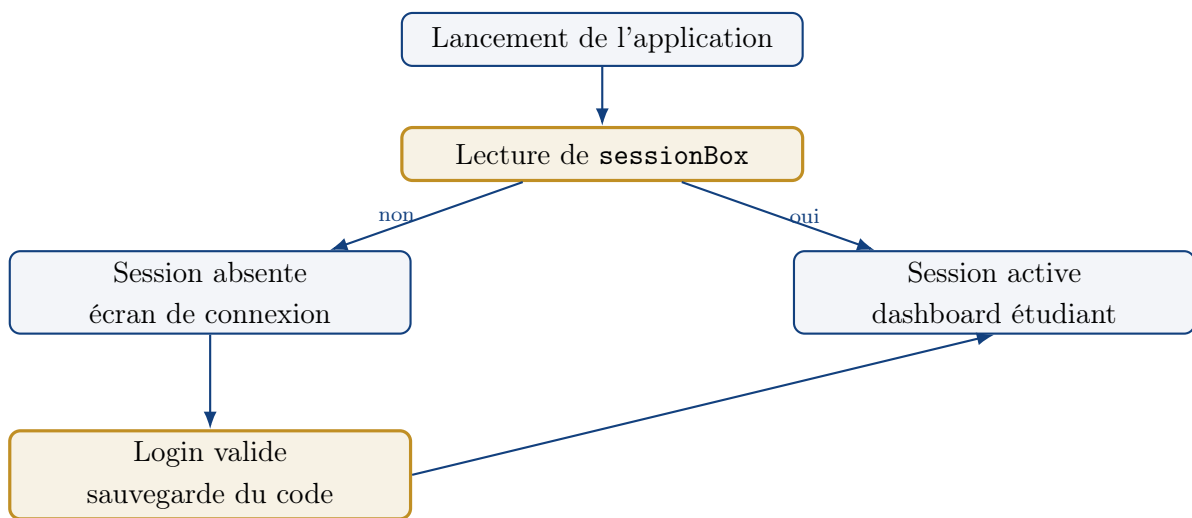


FIGURE 5.2 – Mécanisme d'auto-login basé sur la persistance locale Hive.

Listing 5.4 – Service de persistance de session avec Hive.

```
class SessionService {
  static const String _boxName = 'sessionBox';
  static const String _keyStudentCode = 'studentCode';
  static const String _keyIsLoggedIn = 'isLoggedIn';

  static Future<void> init() async {
    await Hive.openBox(_boxName);
  }

  Future<void> saveSession(String studentCode) async {
    final box = Hive.box(_boxName);
    await box.put(_keyStudentCode, studentCode);
    await box.put(_keyIsLoggedIn, true);
  }

  bool isUserLoggedIn() {
    final box = Hive.box(_boxName);
    return box.get(_keyIsLoggedIn, defaultValue: false) as bool;
  }
}
```

```

}

String? getActiveStudentCode() {
    final box = Hive.box(_boxName);
    return box.get(_keyStudentCode) as String?;
}

Future<void> clearSession() async {
    final box = Hive.box(_boxName);
    await box.clear();
}
}

```

Au démarrage, le routeur de l'application peut interroger ce service avant d'afficher le premier écran. Si `isUserLoggedIn()` retourne `true`, l'étudiant est automatiquement redirigé vers son tableau de bord. Dans le cas contraire, il reste sur le parcours invité et peut accéder à l'écran de connexion. Ce mécanisme améliore l'expérience utilisateur tout en respectant l'approche *Offline-First*, car la décision de navigation ne dépend pas d'une requête réseau.

5.3 Organisation de l'arborescence du projet (dossier lib)

La mise en œuvre pratique de la **Clean Architecture** se traduit directement dans l'organisation des fichiers au sein du dossier principal `lib/`. Afin de maintenir une séparation stricte des responsabilités et de faciliter le travail collaboratif entre les membres du groupe, CampusPulse adopte une structure modulaire claire. Chaque répertoire correspond à un rôle précis : accès aux données, logique métier, interface utilisateur ou services transverses.

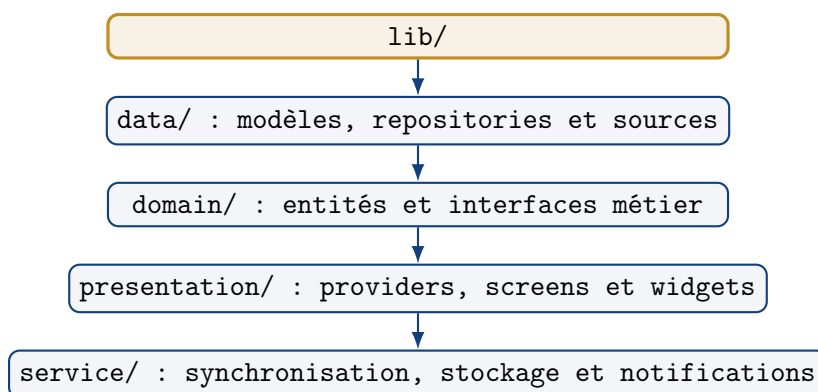


FIGURE 5.3 – Organisation générale du dossier `lib/` dans CampusPulse.

5.3.1 Description des couches principales

La couche `data/` correspond à la couche des données. Elle contient les éléments chargés de communiquer avec Firebase, Hive et les structures JSON manipulées par l'application. Le sous-dossier `models/` regroupe les objets de transfert de données comme `UserModel`, `CourseModel` et `AlertModel`. Ces classes assurent la sérialisation et la désérialisation grâce

aux méthodes `fromMap` et `toMap`. Le sous-dossier `repositories/` contient les implémentations concrètes des accès aux données, tandis que `sources/` regroupe les connexions directes aux passerelles de données brutes, notamment Firebase pour le distant et Hive pour le cache local.

La couche `domain/` représente la couche métier pure. Elle encapsule les entités fondamentales de l'application et les interfaces abstraites des référentiels. Son objectif est de préserver l'indépendance de la logique métier vis-à-vis des frameworks externes. Ainsi, les règles centrales de CampusPulse ne dépendent ni de Firebase, ni de Hive, ni des écrans Flutter.

La couche `presentation/` regroupe toute la partie visible de l'application. Le dossier `providers/` contient les fichiers de gestion d'état basés sur **Riverpod**. Ils distribuent les états des plannings, des sessions et des alertes aux écrans de manière asynchrone. Le dossier `screens/` contient les pages maîtresses, notamment l'accueil invité, l'écran de connexion et le tableau de bord connecté. Enfin, `widgets/` centralise les composants graphiques réutilisables comme les cartes de cours, les boutons personnalisés et les indicateurs de chargement.

5.3.2 Centralisation des services transverses

Le répertoire `service/` permet d'éviter l'éparpillement de la logique technique. Il regroupe les briques système globales utilisées par plusieurs parties de l'application. Cette organisation rend le code plus lisible, plus maintenable et plus simple à tester.

- `course_change_detector_service.dart` : contient l'algorithme d'analyse différentielle qui compare le flux Firebase avec le cache local Hive afin de détecter les changements importants.
- `firebase_service.dart` : initialise et configure la connexion temps réel avec le cloud Google Firebase.
- `local_storage_service.dart` : abstrait l'ouverture, la lecture et l'écriture dans les boîtes de persistance Hive.
- `notification_service.dart` : configure le canal système Flutter Local Notifications pour afficher les alertes directement sur le smartphone de l'étudiant.

Cette organisation du dossier `lib/` renforce la cohérence du projet. Elle permet à chaque membre du groupe d'intervenir sur une zone précise du code sans modifier inutilement les autres couches. Elle prépare également l'application à de futures évolutions, comme l'ajout de nouveaux écrans, de nouvelles sources de données ou de nouveaux types d'alertes.

5.4 Algorithme de détection différentielle des changements de cours

Le coeur de la logique métier de CampusPulse est le service de détection différentielle des changements de cours. Son rôle est de comparer l'état distant de l'emploi du temps, reçu

depuis Firebase, avec l'état local conservé dans Hive. Cette comparaison permet d'identifier les événements qui intéressent réellement l'étudiant : annulation, changement de salle, changement d'horaire ou ajout d'un nouveau cours.

L'intérêt de cette approche est d'éviter les notifications inutiles. L'application ne signale pas chaque synchronisation ; elle alerte uniquement lorsqu'une différence significative est détectée entre l'ancien état et le nouvel état. Le cache Hive devient donc non seulement un support hors ligne, mais aussi une mémoire de référence pour calculer les écarts.

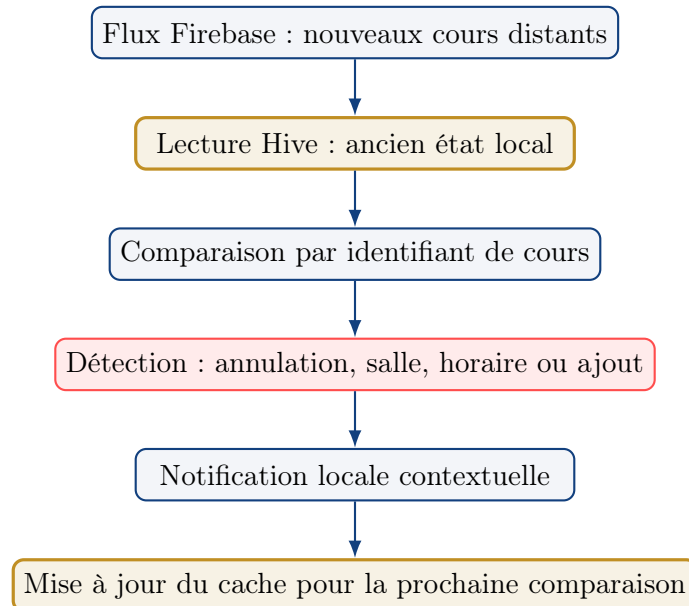


FIGURE 5.4 – Principe de comparaison différentielle entre Firebase et Hive.

L'algorithme s'exécute principalement lors d'un *Pull-to-refresh* manuel ou lorsqu'un flux Firebase `onValue` émet une nouvelle version des données. Il commence par charger le cache local, transforme ensuite les données distantes en objets `CourseModel`, puis compare chaque cours distant à sa version locale. Si l'identifiant existe déjà, le service vérifie les champs sensibles. Si l'identifiant n'existe pas, il considère qu'un nouveau cours a été ajouté.

Listing 5.5 – Service de comparaison différentielle et de synchronisation locale.

```

class CourseChangeDetectorService {
    final Box _courseBox = Hive.box('coursesBox');

    void compareAndSyncWithLocal(
        List<CourseModel> remoteCourses,
        Function(String title, String body) onAlertTriggered,
    ) {
        for (final remoteCourse in remoteCourses) {
            final localCourseData = _courseBox.get(remoteCourse.id);

            if (localCourseData != null) {
                final localMap = localCourseData as Map<dynamic, dynamic>;
                final localClassroom = localMap['classroom']?.toString() ?? '';
                final localStartTime = localMap['startTime']?.toString() ?? '';
                final localIsCanceled = localMap['isCanceled'] == true;
            }
        }
    }
}
  
```

```

    if (remoteCourse.isCanceled && !localIsCanceled) {
        onAlertTriggered(
            'Cours annule',
            'Le cours de ${remoteCourse.title} a ete annule.',
        );
    } else if (remoteCourse.classroom != localClassroom) {
        onAlertTriggered(
            'Changement de salle',
            'Le cours de ${remoteCourse.title} est deplace en salle '
            '${remoteCourse.classroom}.',
        );
    } else if (remoteCourse.startTime.toIso8601String() != localStartTime) {
        onAlertTriggered(
            'Changement horaire',
            'L horaire du cours ${remoteCourse.title} a ete modifie.',
        );
    }
} else if (!remoteCourse.isCanceled) {
    onAlertTriggered(
        'Nouveau cours ajoute',
        'Le cours de ${remoteCourse.title} est planifie en salle '
        '${remoteCourse.classroom}.',
    );
}

_courseBox.put(remoteCourse.id, remoteCourse.toMap());
}
}
}

```

La dernière instruction, `put`, est essentielle : elle écrase l'ancien état local avec la nouvelle version du cours. Ainsi, lors de la prochaine synchronisation, Hive contiendra une base de comparaison actualisée. Cette logique permet à CampusPulse d'être réactif sans devenir bruyant, car l'étudiant reçoit uniquement les alertes qui correspondent à une évolution concrète de son emploi du temps.

5.5 Orchestration entre Firebase, Hive, Riverpod et l'interface

L'implémentation ne se limite pas à des services isolés. Les différentes couches doivent collaborer de manière cohérente. Firebase fournit la source distante, Hive assure la continuité hors ligne, Riverpod expose l'état applicatif aux widgets, et les écrans Flutter affichent les données sous forme de cartes lisibles.

Dans cette organisation, Riverpod joue le rôle d'intermédiaire entre la logique métier et l'interface. Lorsqu'une synchronisation se termine, le fournisseur d'état met à jour la liste des cours. Les widgets qui observent ce fournisseur sont reconstruits automatiquement, sans que l'écran ait besoin de gérer directement les appels réseau ou les accès au stockage local.

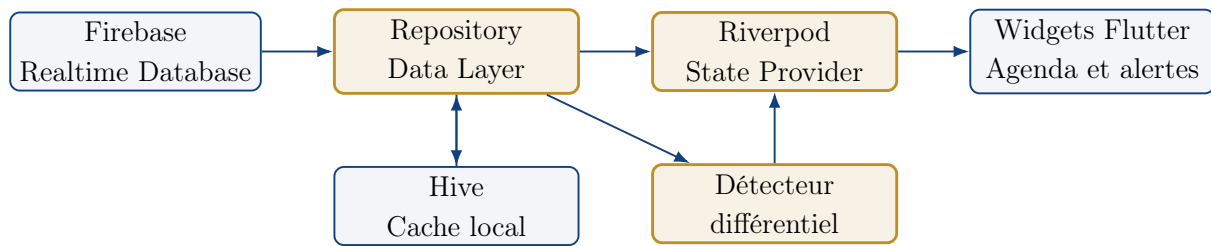


FIGURE 5.5 – Orchestration des couches techniques pendant la synchronisation des cours.

Cette séparation réduit le couplage entre les écrans et les sources de données. L'écran d'emploi du temps ne connaît pas les détails de Firebase, et Firebase ne connaît pas la structure de l'interface. Chaque couche possède une responsabilité claire : récupérer, convertir, comparer, stocker, notifier ou afficher.

5.6 Synthèse de la logique métier implémentée

Le chapitre met en évidence trois choix techniques majeurs. Premièrement, les modèles de données sont conçus pour résister aux anomalies JSON grâce à des conversions explicites et à des valeurs par défaut. Deuxièmement, la session utilisateur est persistée localement avec Hive afin de rendre l'expérience plus fluide grâce à l'auto-login. Troisièmement, l'algorithme de détection différentielle permet de transformer une simple synchronisation de données en un véritable système d'alertes intelligentes.

En combinant ces mécanismes, CampusPulse respecte son orientation **Offline-First**. L'étudiant peut consulter les dernières informations disponibles sans connexion, retrouver automatiquement son espace personnel et recevoir des alertes pertinentes lorsque l'emploi du temps évolue. Cette implémentation constitue donc une base technique solide, maintenable et extensible pour les versions futures de l'application.

5.7 Lien du dépôt GitHub du projet

Afin de permettre la consultation du code source, la vérification de l'organisation du projet Flutter et l'exploration des différentes couches techniques décrites dans ce chapitre, le dépôt GitHub de **CampusPulse** est accessible à l'adresse suivante :

<https://github.com/ganafaye/campuspulse>

Ce lien permet de visiter le code du projet, d'observer la structuration des fichiers, les modèles de données, les services de synchronisation, la gestion locale avec Hive ainsi que l'intégration des écrans Flutter.

Chapitre 6

Réalisation de l'application et scénarios de test

6.1 Approche contextuelle de la réalisation

La phase de réalisation de **CampusPulse** s'est appuyée sur une logique de parcours utilisateur fluide, itérative et robuste. Afin de valider la conformité de l'application avec les exigences académiques de l'Université Alioune Diop de Bambey, nous avons mis en œuvre un scénario de test de bout en bout, appelé *End-to-End*. Ce protocole permet de suivre pas à pas la cinématique réelle d'un utilisateur, depuis sa première ouverture de l'application en mode anonyme jusqu'à l'affichage et la synchronisation complète de ses données métier.

Pour orchestrer et valider visuellement ce flux applicatif sans dépendre d'un parc matériel hétérogène, le groupe a intégré l'outil de simulation **Device Preview**. Configuré au point d'entrée du code, notamment dans `main.dart`, cet utilitaire permet d'observer le rendu des widgets Flutter selon différentes résolutions de smartphones. Il facilite aussi la production de captures homogènes, utiles pour démontrer le comportement réel de l'application pendant la soutenance et dans le présent rapport.

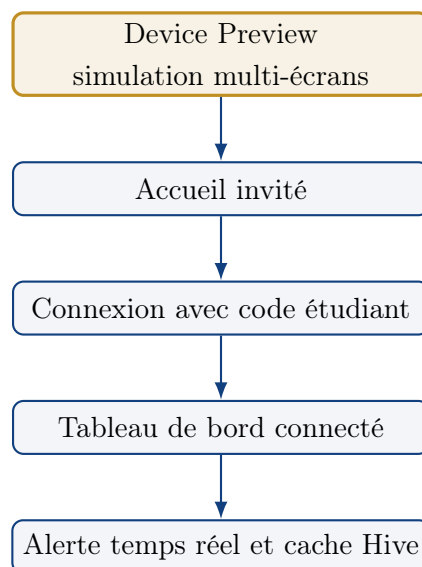


FIGURE 6.1 – Vue synthétique du scénario de test de bout en bout réalisé avec CampusPulse.

6.2 Déroutement chronologique du scénario utilisateur

Le parcours de test suit la trajectoire naturelle de l'étudiant à travers plusieurs interfaces interconnectées. Chaque étape vérifie à la fois une fonctionnalité visible et un mécanisme technique interne : routage, validation de session, persistance locale, mise à jour de l'interface ou réception d'alerte.

6.2.1 Étape 1 : arrivée sur la page d'accueil invité

Au tout premier lancement de l'application, ou lorsque la boîte locale Hive `sessionBox` ne contient aucun identifiant, la couche de routage déclarative `GoRouter` aiguille automatiquement l'utilisateur vers l'**écran d'accueil invité**. Cet écran public sert de vitrine institutionnelle. Il affiche l'identité de CampusPulse et invite l'utilisateur à se connecter, tout en empêchant l'accès aux données sensibles avant authentification.



FIGURE 6.2 – Interface initiale de l'application en mode d'accès public invité.

6.2.2 Étape 2 : transition vers l'écran de connexion

Un appui sur le bouton de connexion déclenche une transition fluide vers l'**écran de login**. Cette interface minimaliste est conçue comme un filtre d'accès. Elle met à disposition de l'étudiant un champ de saisie textuel unique, relié à une logique de validation de

formulaire Dart. L'utilisateur y renseigne son code étudiant avant de soumettre sa demande d'accès.

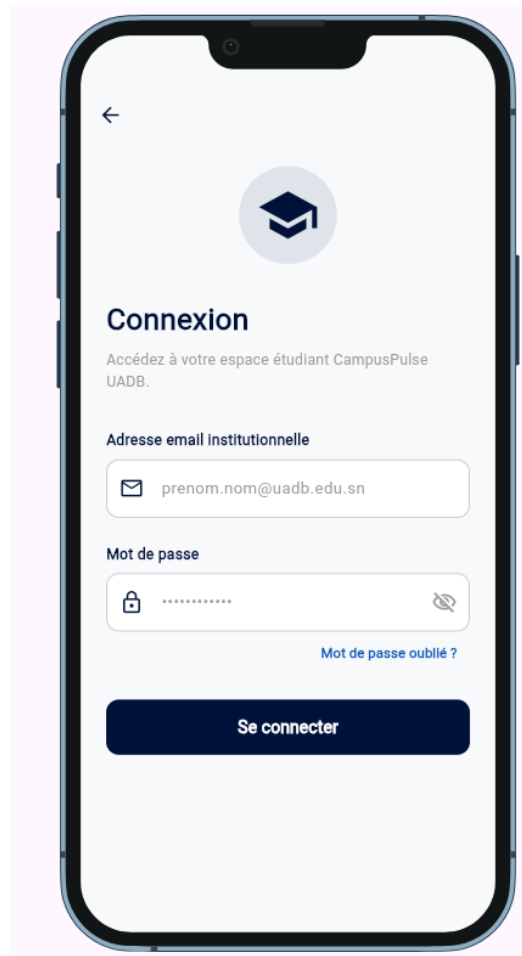


FIGURE 6.3 – Interface d’authentification unique pour la saisie du code étudiant.

6.2.3 Étape 3 : validation de session et redirection vers l’accueil connecté

Dès que l’étudiant valide le formulaire de connexion, deux actions techniques s’exécutent en arrière-plan. Premièrement, le **SessionService** écrit et persiste les clés d’accès, notamment **studentCode** et **isLoggedIn**, dans le stockage local Hive. Deuxièmement, le gestionnaire d’état **Riverpod** notifie le changement de statut, ce qui provoque une redirection immédiate vers l’**accueil connecté**.

Cet écran personnalisé affiche les informations utiles à l’étudiant et donne accès à l’agenda. Le tableau de bord constitue donc le point central de consultation après authentification réussie.

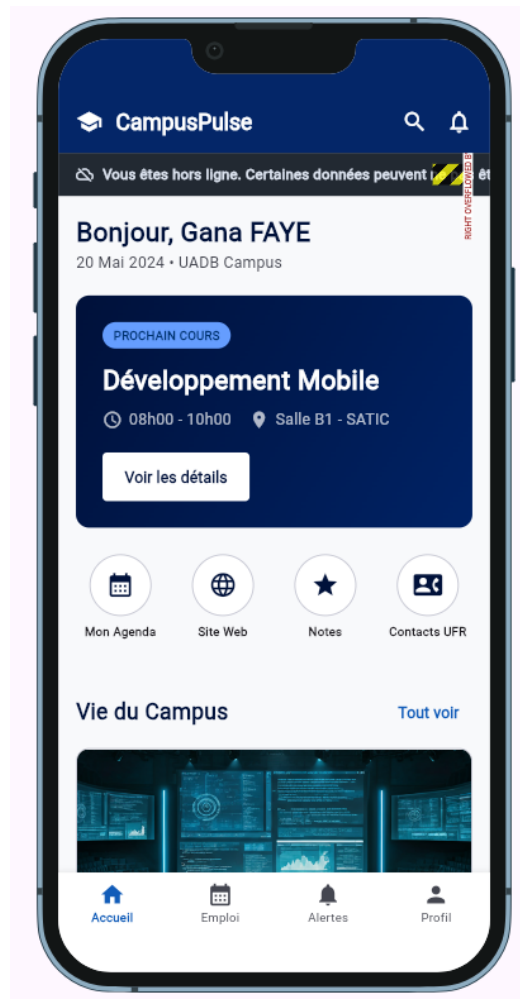


FIGURE 6.4 – Accueil connecté affiché après validation de la session étudiant.

6.2.4 Étape 4 : consultation de l'emploi du temps personnalisé

Après la connexion, l'étudiant peut accéder à l'écran d'emploi du temps. Les cours sont organisés sous forme de cartes lisibles, contenant les informations essentielles : intitulé, enseignant, salle, horaire et niveau concerné. Cette étape valide la capacité de l'application à charger les données métier, à les filtrer selon le profil étudiant et à les présenter dans une interface adaptée au mobile.

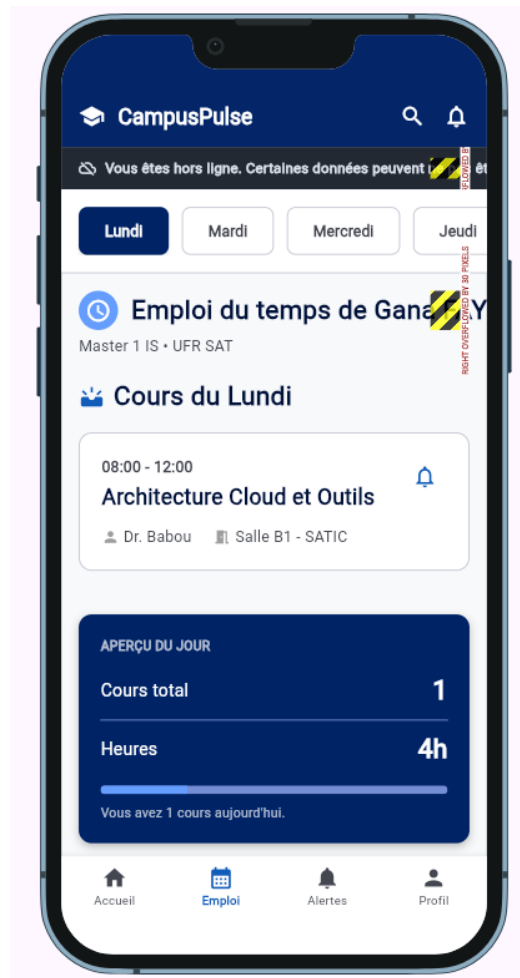


FIGURE 6.5 – Interface de consultation de l’emploi du temps personnalisé de l’étudiant.

6.2.5 Étape 5 : interception d’un changement en temps réel et alerte visuelle

Pour valider l’interconnexion asynchrone avec le serveur cloud, l’application est maintenue ouverte pendant qu’une modification est effectuée dans Firebase Realtime Database. Par exemple, le champ d’un cours peut être modifié pour simuler un changement de salle ou une annonce importante.

L’écouteur de flux **Stream** de la couche de données intercepte cette modification instantanément. L’algorithme **CourseChangeDetectorService** compare la nouvelle valeur avec l’ancien cache local *Hive*, met à jour la persistance *Offline-First* et déclenche l’affichage d’une alerte. L’étudiant reçoit ainsi une information visible, contextualisée et exploitable immédiatement.



FIGURE 6.6 – Réactivité de l’interface lors de l’interception d’une alerte ou modification de planning.

6.3 Bilan de la résilience en mode déconnecté

Afin de clore le scénario de réalisation, le terminal de test a été volontairement isolé de toute connectivité réseau, par exemple par activation du mode avion, avant d’interrompre puis de relancer l’application à froid. Cette manipulation permet de vérifier que l’expérience utilisateur ne dépend pas uniquement de Firebase.

En l’absence de signal internet, l’architecture bascule sur la source de données locale de secours. L’application lit alors le planning directement depuis les données persistées dans Hive, notamment dans `coursesBox`. Ce comportement prouve l’efficacité du mécanisme *Offline-First* développé dans CampusPulse. L’étudiant conserve un accès complet à son dernier emploi du temps synchronisé, sans coupure brutale ni message d’erreur bloquant.

En conclusion, ce scénario de test montre que l’application respecte les exigences principales du cahier des charges : accès contrôlé, session persistante, affichage clair du planning, réception d’alertes et disponibilité des données en mode déconnecté.

Chapitre 7

Mécanisme d'auto-login et persistance de session

7.1 Problématique de l'expérience utilisateur et objectif

Dans le cadre du déploiement d'une application mobile universitaire comme **CampusPulse**, l'accessibilité et la rapidité d'exécution sont des facteurs clés d'adoption par les étudiants. Exiger d'un utilisateur qu'il saisisse manuellement son code étudiant unique à chaque ouverture de l'application représente une rupture d'ergonomie importante. Cette contrainte ralentit l'accès à l'information et crée une surcharge inutile, surtout lorsque l'étudiant souhaite simplement consulter rapidement son prochain cours.

L'objectif de ce module technique est donc d'implémenter un mécanisme d'**auto-login**, c'est-à-dire une connexion automatique après une première authentification réussie. Le système mémorise localement l'état de connexion de l'étudiant et réutilise cette information lors des prochains démarrages. Ainsi, l'application peut décider immédiatement s'il faut afficher l'accueil invité, l'écran de connexion ou rediriger directement l'utilisateur vers son espace connecté.

Ce mécanisme est particulièrement adapté au contexte de CampusPulse, car il ne dépend pas d'une connexion internet active. Même lorsque le réseau est instable ou indisponible sur le campus, l'application peut lire l'état de session depuis le stockage local Hive et offrir une expérience fluide.

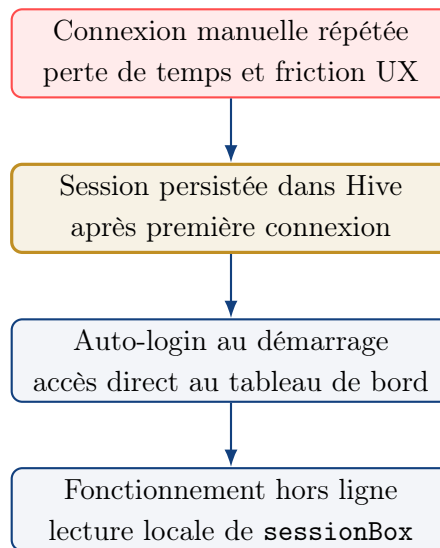


FIGURE 7.1 – Objectif du mécanisme d’auto-login dans l’expérience utilisateur de CampusPulse.

7.2 Architecture technique et cycle de vie de la session

Le fonctionnement de l’auto-login repose sur la collaboration de trois composants majeurs. Le premier composant est **Hive**, utilisé comme base locale NoSQL. À travers une boîte dédiée nommée `sessionBox`, Hive conserve les informations nécessaires à la reprise de session directement sur le smartphone. Le deuxième composant est le `SessionService`, qui sert de couche intermédiaire pour centraliser les opérations de lecture, d’écriture et de suppression des données de session. Le troisième composant est le routeur déclaratif `GoRouter`, qui agit comme un garde de navigation et décide de l’écran à afficher.

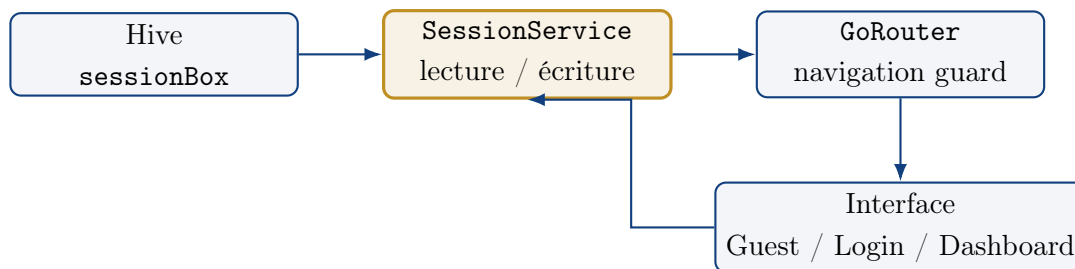


FIGURE 7.2 – Collaboration entre Hive, le service de session et le routeur de navigation.

Le cycle de vie d’une session commence au lancement de l’application. Avant d’afficher l’interface, CampusPulse interroge la clé `isLoggedIn`. Si cette valeur est vraie, l’étudiant est redirigé vers son tableau de bord. Si elle est absente, nulle ou fausse, l’application affiche l’accueil invité puis l’écran de connexion.

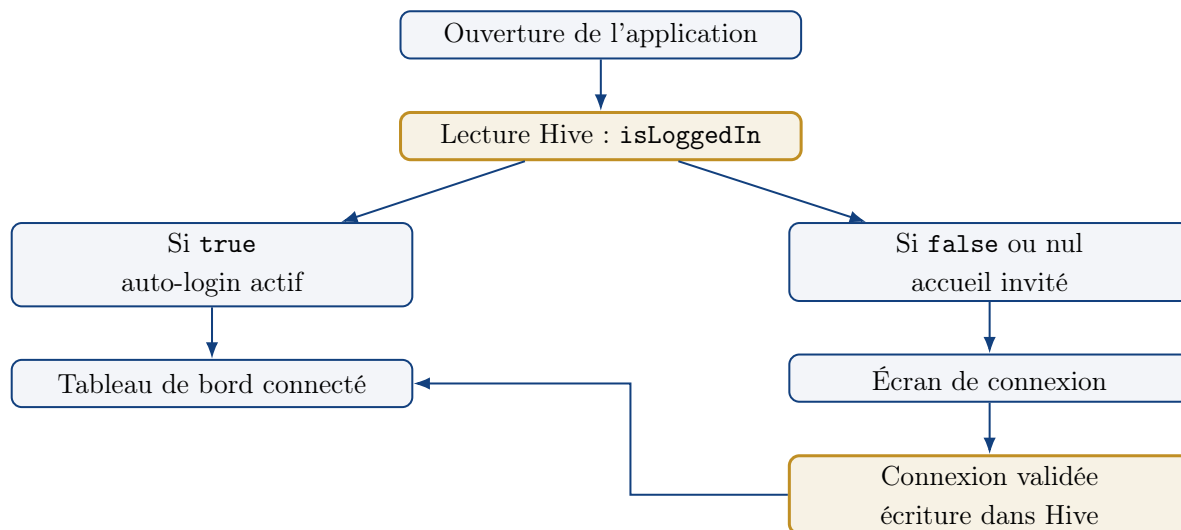


FIGURE 7.3 – Chronologie du cycle de vie d’une session utilisateur.

7.3 Implémentation logicielle et logique d’aiguillage

Pour sécuriser l’initialisation de l’application, l’ouverture de la boîte Hive contenant la session est réalisée au point d’entrée asynchrone du programme, avant le rendu du premier composant Flutter. Cette précaution garantit que le routeur dispose déjà de l’information de connexion lorsqu’il doit choisir la première page.

7.3.1 Persistance des jetons dans le service de session

Dès que l’étudiant valide son identifiant sur l’écran de login, le service fige l’état de connexion en enregistrant deux clés : `studentCode` et `isLoggedIn`. La première clé permet d’identifier l’étudiant et de filtrer les données Firebase. La seconde indique simplement que la session est active.

Listing 7.1 – Écriture de la session après validation du code étudiant.

```

Future<void> openAndWriteSession(String verifiedCode) async {
  final box = Hive.box('sessionBox');
  await box.put('studentCode', verifiedCode);
  await box.put('isLoggedIn', true);
}

```

Cette écriture est asynchrone afin de ne pas bloquer l’interface. Une fois les données enregistrées, l’application peut rediriger l’utilisateur vers l’espace connecté et réutiliser ces informations lors des prochains lancements.

7.3.2 Algorithme d’aiguillage dans le garde de GoRouter

Le cœur du mécanisme d’auto-login réside dans la fonction `redirect` du routeur. Cette fonction agit comme un contrôleur d’accès. Si l’étudiant est déjà connecté et tente de revenir vers l’accueil invité ou l’écran de login, il est automatiquement redirigé vers le tableau de

bord. À l'inverse, si un utilisateur non connecté tente d'accéder à une page privée, il est renvoyé vers l'accueil invité.

Listing 7.2 – Logique d'aiguillage automatique avec GoRouter et Hive.

```
final goRouter = GoRouter(  
  initialLocation: '/',  
  redirect: (context, state) {  
    final box = Hive.box('sessionBox');  
    final bool isLoggedIn = box.get(  
      'isLoggedIn',  
      defaultValue: false,  
    );  
  
    if (isLoggedIn &&  
      (state.matchedLocation == '/guest' ||  
       state.matchedLocation == '/login')) {  
      return '/studentDashboard';  
    }  
  
    if (!isLoggedIn &&  
      state.matchedLocation == '/studentDashboard') {  
      return '/guest';  
    }  
  
    return null;  
  },  
  routes: [  
    // Definition des routes de l'application  
  ],  
);
```

Cette logique améliore à la fois l'ergonomie et la sécurité. Elle évite les connexions répétitives lorsque la session est valide, tout en bloquant l'accès au tableau de bord lorsque l'utilisateur n'a pas encore été authentifié.

7.4 Avantages métiers et résilience Offline-First

L'intégration de l'auto-login via Hive apporte deux bénéfices majeurs au projet **CampusPulse**. Le premier avantage est l'**indépendance réseau**. Contrairement à une authentification classique basée uniquement sur des requêtes serveur, Hive lit le statut de connexion directement depuis la mémoire locale de l'appareil. L'auto-login fonctionne donc même lorsque l'étudiant se trouve dans une zone sans Wi-Fi ou lorsque le smartphone est en mode avion.

Le second avantage concerne la **vitesse d'exécution**. La lecture d'une clé dans Hive est très rapide, car les données sont organisées sous forme de paires clé-valeur optimisées. Pour l'étudiant, cela se traduit par une application réactive qui affiche directement l'espace connecté, sans saisie répétitive et sans écran de chargement prolongé.

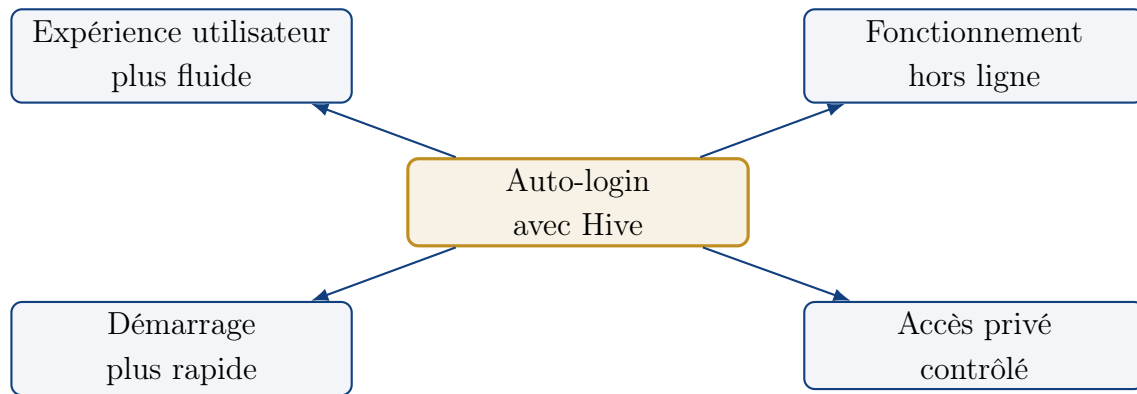


FIGURE 7.4 – Bénéfices fonctionnels et techniques du mécanisme d’auto-login.

En cas de déconnexion volontaire, une fonction de purge est appelée afin d’effacer la session locale. L’application revient alors à son état initial sécurisé et affiche de nouveau l’accueil invité.

Listing 7.3 – Purge de la session lors d’une déconnexion volontaire.

```
Future<void> logout() async {  
    final box = Hive.box('sessionBox');  
    await box.clear();  
}
```

Ainsi, le mécanisme d’auto-login renforce la cohérence générale de CampusPulse. Il assure une continuité d’utilisation, réduit les frictions à l’ouverture de l’application et reste pleinement compatible avec l’approche *Offline-First* défendue tout au long du projet.

Chapitre 8

Conclusion et perspectives

Le projet **CampusPulse** a permis de concevoir et de réaliser une application mobile orientée vers la gestion de l’emploi du temps universitaire et la diffusion d’alertes en temps réel. À travers l’analyse des besoins, la conception UI/UX, les choix technologiques et l’implémentation technique, le projet répond à une problématique concrète rencontrée par les étudiants : accéder rapidement à une information pédagogique fiable, même lorsque la connexion internet est instable.

Sur le plan technique, l’application s’appuie sur **Flutter** pour l’interface mobile, **Firebase Realtime Database** pour la synchronisation distante, **Hive** pour la persistance locale et **Riverpod** pour la gestion d’état. L’approche *Offline-First* constitue l’un des apports majeurs de la solution, car elle garantit la consultation du dernier planning synchronisé en mode déconnecté. Les modèles de données, les services transverses et l’organisation du dossier `lib/` montrent également une volonté de maintenir un code structuré, évolutif et adapté à un travail collaboratif.

Les scénarios de test présentés démontrent que le parcours utilisateur principal est fonctionnel : arrivée sur l’accueil invité, authentification, accès à l’espace connecté, consultation de l’emploi du temps, réception d’alertes et continuité de service hors ligne. Ces résultats confirment la pertinence du MVP réalisé dans le cadre du module de développement mobile.

Comme perspectives, CampusPulse pourra intégrer une interface d’administration plus complète, une authentification institutionnelle renforcée, un système de rôles pour les responsables pédagogiques, ainsi qu’un historique avancé des changements de planning. L’application pourra aussi être enrichie par des notifications personnalisées, une synchronisation plus fine par filière et niveau, et une publication progressive sur les plateformes Android et iOS.

En définitive, CampusPulse constitue une base solide pour moderniser la communication académique autour des emplois du temps et améliorer l’expérience quotidienne des étudiants de l’UADB.